Perceiver & Perceiver IO

Implementation and Experimental Analysis

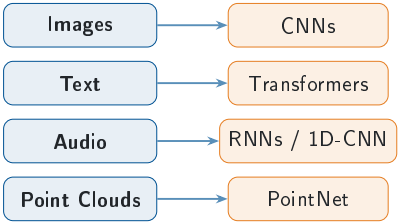
Francesco Gigli e Corso Vignoli

Università degli Studi di Firenze
Course B031278 — Deep Learning

Agenda

The Problem: Modality-Specific Architectures

One modality, one architecture



Each data type is usually paired with a dedicated model and domain-specific assumptions about structure.

The scaling bottleneck

Self-attention cost

$$\mathcal{O}(M^2 \cdot d)$$

grows quadratically with input length M

Input representation	M
16×16 image patches	256
raw 224×224 pixels	50,176

Raw pixels make standard attention impractical.

Core question: can one architecture handle arbitrary inputs without quadratic cost?

The Perceiver Solution

Core idea: learned latents read the input once; expensive processing stays in latent space.

1. Read

Cross-attention

M inputs $\rightarrow$ $N \ll M$ latents

2. Process

Latent block

transform N latent slots

3. Reuse

Shared weights

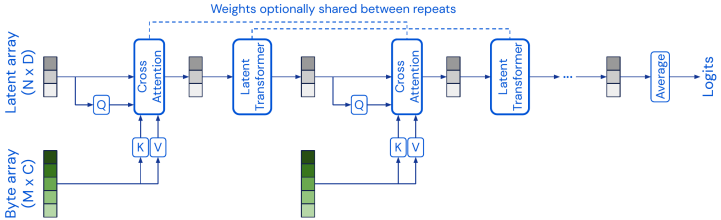
repeat the same processor

Complexity shift

Read the full input once; repeat the expensive work in the compact latent space $N \ll M$.

Operation	Cost
Read input once	$\mathcal{O}(M \cdot N)$
Process latents for L layers	$\mathcal{O}(L \cdot N^2)$
Perceiver	$\mathcal{O}(MN + LN^2)$

Perceiver Architecture



Encode and process

- $X \in \mathbb{R}^{M \times C}$ supplies keys and values.
- $Z \in \mathbb{R}^{N \times D}$ supplies queries, with $N \ll M$.
- Cross-attention maps $X \to Z$: $\mathcal{O}(MN)$.

Latent core and output

- Latent Transformer updates Z only.
- Repeated blocks may share weights.
- Original Perceiver pools Z; Perceiver IO uses output queries.

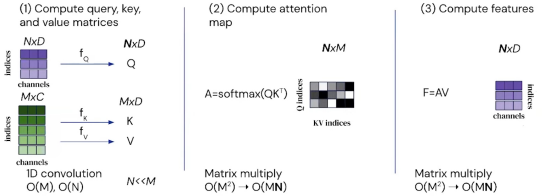
Cross-Attention: The Key Mechanism

Formally:

$$Q = X_{\text{latent}}\, W_Q \in \mathbb{R}^{N \times d_k}$$
$$K = X_{\text{input}}\, W_K \in \mathbb{R}^{M \times d_k}$$
$$V = X_{\text{input}}\, W_V \in \mathbb{R}^{M \times d_v}$$

$$\text{CrossAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

The latents are the queries, the input provides keys and values $\Rightarrow$ informational *bottleneck*.

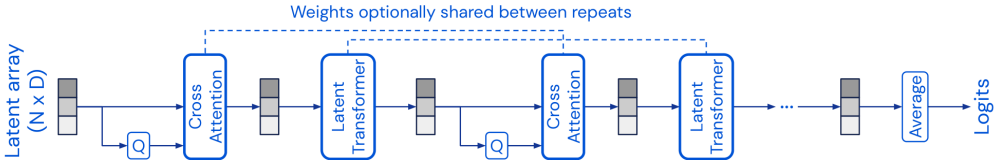


Key insight:

- Attention matrix: $N \times M$ (not $M \times M$)
- Complexity: $\mathcal{O}(N \cdot M \cdot d_k)$
- Decouples depth from input size

Latent Transformer & Weight Sharing

Once information is in the latent array, depth is spent by reusing the same processor on a compact state.



Weight sharing

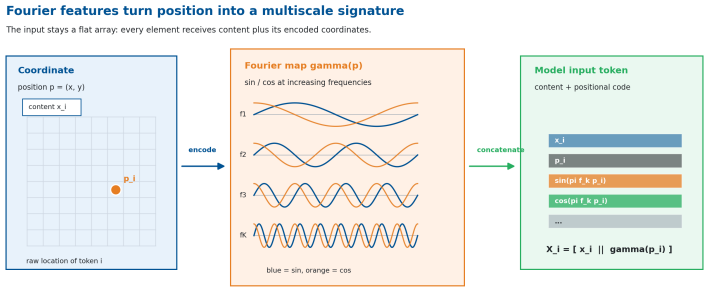
- The same Latent Transformer block B_θ is reused at every repeat, with shared parameters θ.
- Unrolling depth behaves like an RNN over the latent state: $Z_{t+1} = B_\theta(Z_t)$.

Why it matters

- Iterative refinement without instantiating a new processor at each step.
- Parameter count stays fixed as depth grows; computation stays in the N-slot latent space.

Fourier Positional Encoding

Perceiver receives a flat array, so location has to be part of each input token.



General recipe

Use domain coordinates: images (u, v), point clouds (x, y, z), or text index t; sine/cosine bands encode them at multiple scales.

Input seen by attention

The array is still flat, but each token is enriched before projection: $X_i = [\, x_i \,\|\, \gamma(p_i) \,]$.

Perceiver Forward Pass: Shape Checkpoints

Data flow in the original Perceiver:

1. Input array: each token stacks content x with a Fourier position code $\gamma(\mathrm{p})$,

$$X = [\,\mathrm{x} \,\|\, \gamma(\mathrm{p})\,] \in \mathbb{R}^{M \times C}.$$

2. Learned latent array:

$$Z_0 \in \mathbb{R}^{N \times D}, \qquad N \ll M$$

3. Cross-attention reads input:

$$Q = ZW_Q, \quad K = XW_K, \quad V = XW_V$$

4. Latent Transformer applies self-attention + FFN only on N latents.

Shape checkpoints:

Object	Shape	Meaning
X	$M \times C$	input array (content+pos)
Z_0	$N \times D$	learned latents
Q	$N \times d_k$	latent queries
K, V	$M \times d_k, d_v$	input keys/values
A	$N \times M$	attention map

Core idea

The input size M affects only the cross-attention read; depth is spent in the compact latent space.

Original Perceiver Output: Pooling Classifier

Fixed-shape output head:

$$Z_T = \mathrm{Encoder}(X, Z_0)$$

$$z = \frac{1}{N}\sum_{i=1}^{N} Z_T[i]$$

$$\mathrm{logits} = zW_{\mathrm{cls}} + b_{\mathrm{cls}}$$

- Mean pooling collapses all latents to one vector
- The classifier predicts one global label
- This is ideal for classification, not dense structured outputs

Where FFN and sharing fit:

- Each latent block is pre-norm attention, residual, FFN, residual
- FFN adds non-linearity after attention mixing
- Weight sharing repeats the processor over iterations

Fixed-shape output

Pooling collapses the N latents into a single vector, so this head emits one global prediction per input.

Computational Complexity Analysis

Model	Self-Attn	Cross-Attn	Total
Standard Transformer	$\mathcal{O}(M^2 d)$	—	$\mathcal{O}(M^2 d \cdot L)$
Perceiver	$\mathcal{O}(N^2 d)$	$\mathcal{O}(MNd)$	$\mathcal{O}((MN + N^2 L)\, d)$

Symbols:

- M: number of input elements
- N: number of latent slots, with $N \ll M$
- L: number of latent processing layers
- d: hidden channel dimension

Interpretation:

- The full input is read through cross-attention
- Repeated self-attention happens only in latent space
- If N is fixed, depth becomes independent of input length
- Concrete sizes are evaluated in the experiments

GELU Activation & LAMB Optimizer

GELU (Gaussian Error Linear Unit):

$$\mathrm{GELU}(x) = x \cdot \Phi(x) \approx x \cdot \sigma(1.702\,x)$$

- Smooth activation used in Transformer architectures
- Allows small negative gradient flow

LAMB (Layer-wise Adaptive Moments):

$$r_t^{(l)} = \frac{\|w_t^{(l)}\|}{\|w_t^{(l)} + \eta\,\hat{u}_t^{(l)}\|}$$

Per-layer trust ratio rescales updates and stabilizes large-batch training.

Why LAMB for Perceiver?

- Cross-attention bottleneck creates high variance gradients
- Standard Adam can destabilize with large batches
- LAMB safely scales batch sizes up to 1024

Optimizer	Large Batches	Layer-wise LR
Adam	Diverges	Global
LAMB	**Stable**	**Per-layer trust**

Implementation

Aspect	Paper implementation	Project code
Core model	Cross-attention into learned latents, latent self-attention, FFN, weight sharing	Same modules; weight sharing is exposed as an ablation switch
Model scale	Large vision/language runs: up to 512×1024 latents and 201M-param IO	Smaller runs: CIFAR-10 96×384, ModelNet40/text 128×512
Data setting	ImageNet, ModelNet40, large language pretraining corpora	CIFAR-10, ModelNet40, WikiText-103 byte-level MLM, GLUE fine-tuning
Training budget	Distributed accelerators, batches up to 512–1024	Single-GPU budget, batches 32–128, LAMB retained for stability
Practical additions	Minimal regularization in the reference setup	Dropout, logging, attention-map extraction, and reproducible ablations

The implementation keeps the architectural mechanism intact; the main differences are scale, data budget and training resources.

Implementation Differences from the Paper

Choice	Original paper (ImageNet)	This project
Input tokenization	Raw pixels, no patching ($M = 50{,}176$)	2×2 patch embedding (CIFAR-10 $M = 256$)
Latent array $N \times D$	512×1024	96×384 (CIFAR-10), 128×512 (ModelNet40)
Cross-attend iterations T	8	4 (CIFAR-10), 2 (ModelNet40)
Positional encoding	2D Fourier, $K = 64$ bands/axis (258 dims)	2D Fourier, 64 dims
Weight sharing	On by default (shared across T)	Exposed as an ablation switch (on/off)
Parameters	30–200M+	3.35M (CIFAR-10) – 10M (IO)

Kept identical to the paper

Core mechanism preserved: cross-attention bottleneck → weight-shared latent transformer → mean-pooling (Perceiver) / output-query decoder (IO); GELU MLP, pre-norm LayerNorm, Fourier frequencies linearly spaced to Nyquist.

CIFAR-10: Experimental Setup

Goal: validate the Perceiver on flattened pixel patches without any convolutional assumptions.

Hyperparameter	**Value**
Latent dim ($N \times D$)	96×384
Cross-attn stages	4
Transformer blocks	4 (shared)
Attention heads	3
Patch size	2×2
Dropout	0.2
Optimizer	LAMB
Learning rate	0.004
Batch size	64
Epochs	120
Total parameters	**$\sim$3.35M**

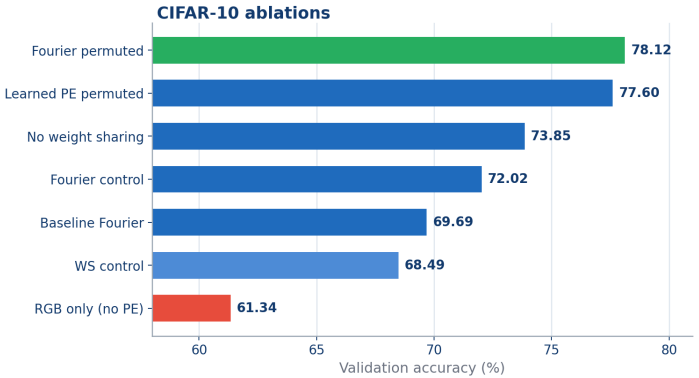
Ablation study plan:

1. Impact of positional encoding
2. Weight sharing vs no sharing
3. Robustness to pixel permutation
4. Best classifier-head configuration

Data pipeline:

- 32×32 RGB images split into 2×2 patches ($M = 256$ tokens, 12 raw channels each)
- 2D Fourier positional encoding (64 features $\rightarrow$ input dim 76)
- Augmentation: RandAugment (2 ops, magnitude 9)

CIFAR-10: Ablation Accuracy



Measured comparisons

- Best Perceiver run: **78.12%**
- No PE vs Fourier control: **−10.68pp**
- No sharing vs WS control: **+5.36pp**
- Fourier vs learned PE under fixed permutation: **+0.52pp**

Source

Values from the local experiment report: `experiment_summary.csv`.

CIFAR-10: Ablation Matrix

Run	PE	Perm.	Sharing	Params	Acc.
Fourier permuted	Fourier	yes	yes	3.35M	**78.12%**
Learned PE permuted	learned	yes	yes	3.35M	77.60%
No weight sharing	Fourier	no	no	8.67M	73.85%
Fourier control	Fourier	no	yes	3.35M	72.02%
Baseline Fourier	Fourier	no	yes	3.35M	69.69%
WS control	Fourier	no	yes	3.35M	68.49%
RGB only	none	no	yes	3.30M	61.34%

Position signal

Removing positional features costs **10.68pp** against Fourier control.

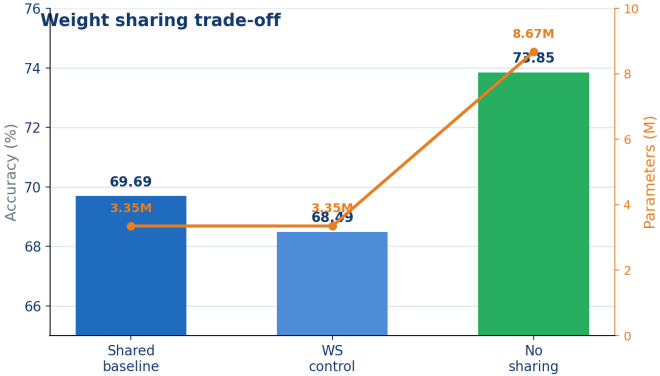
Capacity

Removing sharing adds **5.32M** parameters and gives **+5.36pp**.

Permutation

Fixed permutation keeps the task learnable when coordinates are provided.

CIFAR-10: Weight Sharing Trade-off



Config	Params	Acc.
Shared baseline	3.35M	69.69%
WS control	3.35M	68.49%
No sharing	8.67M	73.85%

Measured delta

- Accuracy: $+$**5.36pp**
- Parameters: $+$**5.32M**
- Multiplier: **2.59**$\times$

$$73.85 - 68.49 = 5.36$$

CIFAR-10: Fixed-Permutation Test

Experiment: train *and* evaluate on pixels shuffled by a single fixed permutation σ (same σ throughout).

Configuration	Acc.
Fourier PE (natural order)	69.69%
Fourier PE (permuted)	**78.12%**
Learned PE (permuted)	77.60%

Measured result:

- Permuting pixels raises accuracy: +8.43pp
- Fourier vs learned (permuted): +0.52pp

Config note: the learned-PE run used a single cross-attention stage (vs 4 in the other ablations); same parameter count.

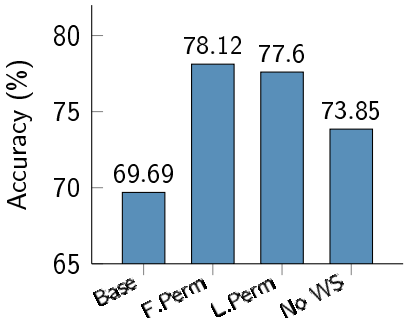
Interpretation:

- The Perceiver has **no built-in 2D bias**: position is read only from the PE
- With a fixed σ in train *and* test, it learns the layout from the Fourier features alone
- Fourier PE remains slightly ahead of learned PE in this setting

Architectural implication

The input order is not hard-coded; ordering information enters through positional features.

CIFAR-10: Best Perceiver Configuration



Best Perceiver result:

Configuration	Acc.
Fourier PE, permuted	**78.12%**
Learned PE, permuted	77.60%
No weight sharing	73.85%
Baseline Fourier	69.69%

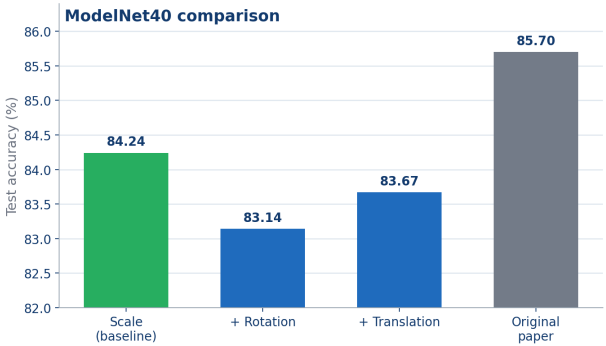
Result pattern:

- Fourier PE carries the spatial signal explicitly.
- Weight sharing is an efficiency–accuracy trade-off.

ModelNet40: Setup & Results

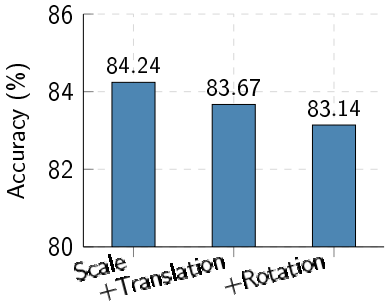
Configuration:

Hyperparameter	Value
Latent dim ($N \times D$)	128×512
Cross-attn stages	2
Transformer blocks	6 (shared)
Points per object	2048
Optimizer	LAMB
Epochs	200
Batch size	128



Closest paper comparison: 84.24% vs 85.70% (gap **1.46pp**).

ModelNet40: Augmentation Effects



Measured deltas:

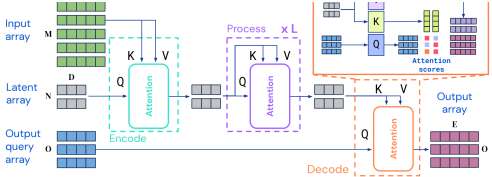
- Scale baseline: **84.24%**
- + translation: **−0.57pp**
- + rotation: **−1.10pp**

Likely factors:

- 3D Fourier PE already exposes coordinates
- ModelNet40 objects are already centred
- Extra variation did not improve this run

Paper comparison: 85.70%
Gap: **1.46pp** from the scale baseline.

Perceiver IO: Structured Outputs



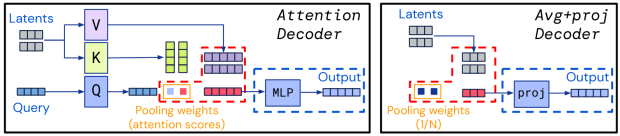
Extension over Perceiver (Jaegle et al., ICLR 2022):

- **Encode**: input $\rightarrow$ latent (same cross-attn encoder)
- **Process**: latent self-attention ($\times L$)
- **Decode**: output queries read latents

Why it matters:

- Original Perceiver: one pooled global output
- IO: task-specific output shape
- Same latent bottleneck, more general decoder

Perceiver IO Decoder with Output Queries



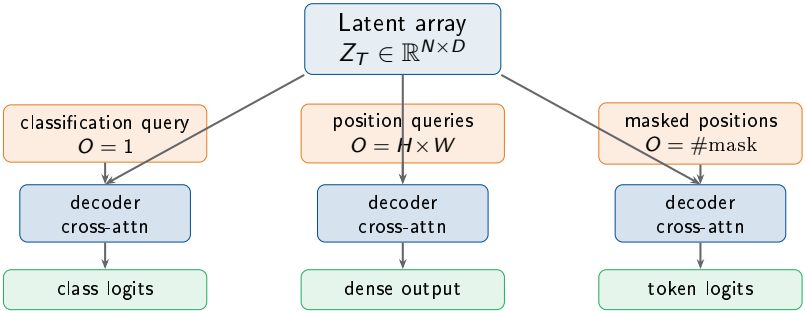
Decoder cross-attention (single layer):

$$Q_{\text{dec}} = \text{OutputQuery}\, W_Q, \quad K_{\text{dec}} = \text{Latent}\, W_K, \quad V_{\text{dec}} = \text{Latent}\, W_V$$

$$\text{Output} = \text{softmax}\left(\frac{Q_{\text{dec}} K_{\text{dec}}^{\top}}{\sqrt{d_k}}\right) V_{\text{dec}} \in \mathbb{R}^{O \times E}$$

- The number of output queries O is **task-dependent** and independent of N and M
- A single decoder cross-attention layer suffices (no self-attention among outputs)

Output Query Design per Task



Classification

One learned query produces one global prediction.

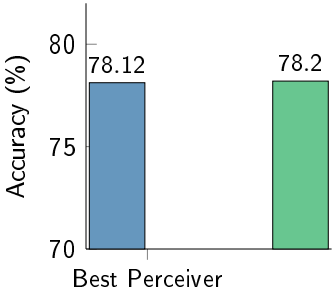
Dense outputs

Output shape is controlled by the query array.

Masked LM

Only masked positions need decoder queries.

CIFAR-10: Perceiver vs Perceiver IO



Decoder effect:

- Output query is a learned aggregation instead of fixed mean pooling
- Decoder cross-attention attends selectively to relevant latents
- Small gain on CIFAR-10, but same mechanism enables MLM and dense outputs
- Cost: $\sim 1.6\times$ parameters (5.22M vs 3.35M)

Model	**Acc.**	**Δ**
Best Perceiver	78.12%	—
Perceiver IO	**78.20%**	**+0.08pp**

WikiText-103: Masked Language Model Pre-training

Goal: pre-train a general language encoder without specialized tokenizers.

Component	Detail
Tokenization	Byte-level (UTF-8)
Vocabulary size	256
Sequence length	1024
Mask probability	15%
Model	Perceiver IO ($\sim$10M)
Epochs / batch	50 / 32
Masked byte acc.	**82.20%**

Why byte-level?

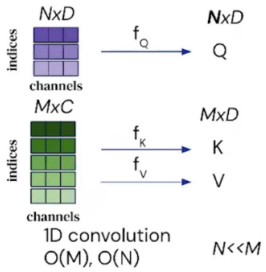
- No BPE/WordPiece tokenizer needed
- Truly language-agnostic input
- Fixed vocabulary of size 256
- Trade-off: longer sequences for same text

MLM task:

- 15% of byte positions masked
- Output queries = masked positions
- Decoder predicts original byte values
- Cross-entropy loss over 256 classes

GLUE: Fine-tuning on 8 Tasks

Transfer pipeline:



Fine-tuning setup:

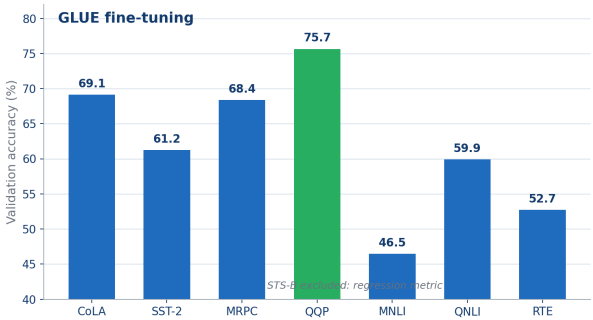
- Transfer pre-trained MLM encoder
- Replace output queries with classification query
- 30 epochs, LR $= 5 \times 10^{-4}$
- Byte-level input (same as pre-training)

8 GLUE benchmark tasks:

- **Single sentence:** CoLA, SST-2
- **Similarity:** MRPC, STS-B, QQP
- **Inference:** MNLI, QNLI, RTE

Each task tests a different NLU capability with the same architecture.

GLUE: Per-Task Results



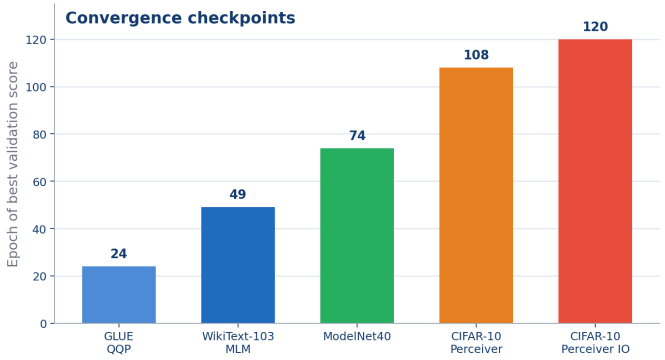
Per-task pattern:

- **QQP** (paraphrase): best task at 75.65%
- **CoLA** 69.13%, **MRPC** 68.38%
- **MNLI** 46.47%, **RTE** 52.71%
- Classification mean: ∼62%

Setup factors:

- Byte-level input, no subword tokenizer
- Smaller model: ∼10M vs 201M
- STS-B excluded: regression metric

Convergence: Representative Checkpoints



Best validation epoch:

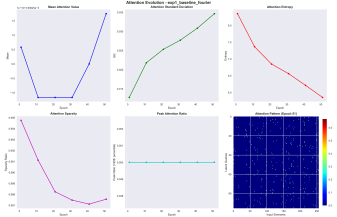
Run	Epoch
GLUE QQP	24
WikiText-103 MLM	49
ModelNet40	74
CIFAR-10 Perceiver	108
CIFAR-10 Perceiver IO	120

Pattern:

- Fine-tuning peaks earlier than image training
- Point clouds converge before CIFAR-10
- CIFAR-10 requires the longest schedule

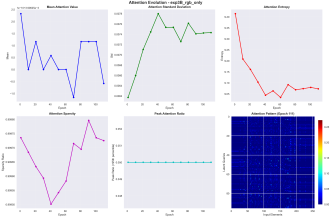
Attention Maps Analysis

Fourier PE (baseline, final maps)



Structured, spatially selective patterns (entropy ∼5.9). Latents specialize in different image regions.

RGB Only — no PE (final maps)



Diffuse, near-uniform patterns (entropy ∼9.1). No spatial selectivity — the model cannot localize information.

Interpretation

With Fourier PE, cross-attention learns spatially coherent patterns across training. Without PE, attention entropy remains high and patterns lack structure.

Comprehensive Results Summary

Modality	Model	Best Acc.	Paper	Experiments
Images (CIFAR-10)	Perceiver	78.12%	>90%	7 ablations
Images (CIFAR-10)	Perceiver IO	**78.20%**	—	1
Point Clouds (ModelNet40)	Perceiver	84.24%	85.7%	3 augmentations
Text (WikiText-103)	Perceiver IO	82.20% MLM	—	1
NLU (GLUE, 8 tasks)	Perceiver IO	∼62% avg	—	8 fine-tuning

Measured results:

- ModelNet40: 1.46pp gap from paper
- Perceiver IO: 78.20% vs best Perceiver: 78.12%
- MLM pre-training transfers to GLUE fine-tuning

Overall numbers:

- ∼2500 lines of PyTorch code
- 20+ distinct training runs
- 3 modalities: 2D images, 3D points, text
- 100% reproducible via `reproduce.py`

Gap vs Original Papers

ModelNet40 (Point Clouds):

Source	Acc.
Original Paper	85.7%
This implementation	84.24%
Gap	**1.46pp**

Small reproduction gap.
Main scale difference: batch size 128 vs 512.

CIFAR-10 (Images):

Source	Acc.
Original Paper	>90%
This Perceiver IO	78.20%
Gap	**∼12pp**

Larger image gap.
Main scale difference: distributed TPU training vs single GPU.

Scale factors

The CIFAR-10 gap is primarily due to **batch size** (64 vs 1024), **latent count** (96 vs 256), and **compute time** limitations. The Perceiver architecture is particularly sensitive to these scaling factors on image data.

Hardware Limitations & Design Choices

Parameter	Original Paper (Google)	Project Setup
Hardware	512 TPU v3 cores	1 NVIDIA consumer GPU
Batch size	up to 1024	max 64–128
Latents (N)	256–512	96–128
Model size	30–100M+ params	3.35–10M params
Training time	days (distributed)	hours–days (single GPU)

Consequences:

- LAMB optimizer less effective with small batches
- Fewer latents limit representational capacity
- Reduced augmentation due to time constraints
- Fewer hyperparameter search iterations

Mitigation strategies:

- Weight sharing to reduce memory
- Gradient accumulation where possible
- Careful cosine annealing schedule
- Focus experiments on most informative ablations

Possible Improvements

Architecture:

- Increase latent count ($N = 256+$) with gradient checkpointing
- Add multi-scale cross-attention (coarse $\rightarrow$ fine)
- Explore relative positional encodings (RoPE, ALiBi)
- Deeper decoder for structured outputs

Training:

- Larger batch sizes with gradient accumulation
- Mixed precision (FP16/BF16) training
- Longer training schedules
- Advanced augmentation (CutMix, MixUp)

Data & Tasks:

- ImageNet pre-training for vision
- Subword tokenization for NLP
- Multi-modal joint training
- Audio modality experiments

Analysis:

- Layer-wise attention probing
- Latent space clustering visualization
- Systematic hyperparameter search
- Comparison with efficient Transformers (Linformer, Performer)

Empirical Summary

Observed strengths:

- Modality-agnostic architecture
- Weight sharing: $\sim 2.6\times$ fewer parameters, modest accuracy cost
- Perceiver IO slightly edges base Perceiver on CIFAR-10
- MLM pre-training transfers to GLUE fine-tuning
- Fourier PE is robust and general-purpose

Observed limitations:

- Performance gap vs paper on images ($\sim$12pp)
- LAMB optimizer sensitive to batch size
- Byte-level tokenization produces very long sequences for text
- Augmentation can hurt with limited training
- Hyperparameter tuning is time-consuming

Central empirical finding

Positional encoding is the largest single factor in the CIFAR-10 ablation: removing it changes accuracy from 72.02% to 61.34%.

Conclusions

Architecture properties:

- Single architecture for images, point clouds, and text
- Linear complexity in input size
- Memory efficiency via weight sharing ($\sim$61%)
- Zero CNN/RNN domain-specific assumptions
- Output queries enable flexible structured decoding

Remaining limitations:

- Requires large compute to match specialized models
- Heavily depends on positional encoding
- Outperformed by efficient CNNs on standard vision benchmarks
- LAMB optimizer introduces training instability at small scale
- Latent bottleneck limits fine-grained spatial reasoning

Closing statement

The experiments show that a **single general-purpose architecture** can be applied to images, point clouds, and text. At the tested scale, performance depends strongly on positional encoding, latent capacity, and available training compute.

References

- A. Jaegle et al., *Perceiver: General Perception with Iterative Attention*, ICML 2021.
- A. Jaegle et al., *Perceiver IO: A General Architecture for Structured Inputs & Outputs*, ICLR 2022.
- A. Vaswani et al., *Attention Is All You Need*, NeurIPS 2017.
- Y. You et al., *Large Batch Optimization for Deep Learning: Training BERT in 76 Minutes*, ICLR 2020.
- M. Tancik et al., *Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains*, NeurIPS 2020.
- N. Shazeer, *GLU Variants Improve Transformer*, arXiv 2020.

Appendix: Expected Questions

Architecture:

1. Why cross-attention instead of full self-attention?
   → Reduces $\mathcal{O}(M^2)$ to $\mathcal{O}(NM)$
2. How does weight sharing affect performance?
   → -5.36pp for $\sim 2.6\times$ fewer params
3. Why Fourier PE over learned PE?
   → More robust to permutation, generalizes better
4. What is the role of GEGLU?
   → Higher non-linear capacity, stable gradients

Experiments:

5. Why is the CIFAR-10 gap so large?
   → Hardware limits: batch 64 vs 1024, 96 vs 256 latents
6. Why byte-level tokenization for NLP?
   → Truly modality-agnostic, no external tokenizer
7. How does Perceiver IO improve on Perceiver?
   → Learned output queries vs mean pooling
8. Could this replace ViT/BERT?
   → Not yet at limited scale; promising at Google scale

Thank you!

Questions?

Francesco Gigli e Corso Vignoli

Code and models available upon request

Code:	∼2500 lines PyTorch (from-scratch)
Experiments:	20+ training runs, 3 modalities
Reproducibility:	`python reproduce.py`